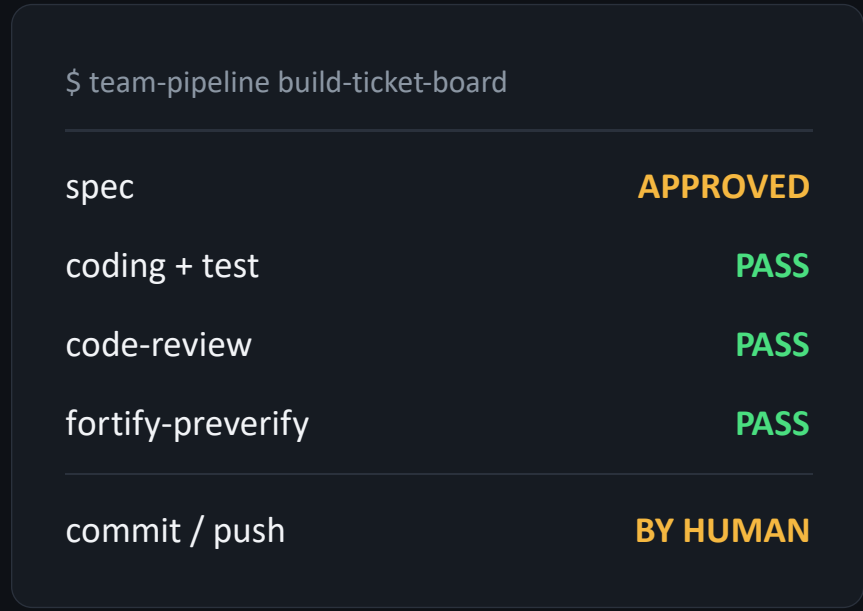
> HWDC 2026 · SESSION 5021

從寫得快到交付可信的 AI Coding 工程戰場

有依據的 **Spec** ＋ 會擋人的 **Harness**
把真人團隊的交付流程，翻譯成 **Agent Team**

陳葵懋 Ian Chen
一等一科技 研發部主任工程師 · Microsoft AI MVP · MCT



> SPEAKER · 講者介紹



陳葵懋

Ian Chen

一等一科技 研發部主任工程師
Microsoft Certified Trainer (MCT)

15 YEARS
MICROSOFT AI MVP

iangithub.github.io

企業 AI 與軟體架構實務

軟體設計與開發經驗，長期專注 .NET、Azure、容器化與生成式 AI，擅長將企業系統、雲端平台與 AI 技術整合為可上線的解決方案。

20+ 年軟體開發經驗

1,000+ 培訓學員

百萬+ Chatbot 使用者

6+ 本共同著作

專長領域

Azure OpenAI | 生成式 AI | RAG | AI Agent | MCP | .NET / C# | Azure / DevOps | Docker / Kubernetes | Agent Framework | 企業系統整合

演講與培訓

- 研討會講者：Microsoft TechDays、DevDays Asia、Modern Web、Hello World、MOPCON、iThome AI & Cloud
- 企業、公務機關與大學院校 AI 技術培訓（司法院、國教署）

代表成果

- 疫情期間開發「敗口罩」與「找找快篩試劑」LINE Chatbot，兩項專案累計服務百萬以上使用者
- 共同著作 6+ 本：ChatGPT、LangChain、AI Agent、MCP

軟體交付的自動化，又往前推了一段

過去

手動交付

人工建置、人工測試、人工部署

→

CI/CD

自動化交付

建置、測試、部署交給 pipeline；
程式仍由人寫、人審

→

現在

自動化撰寫程式碼

Agent 寫程式、跑測試、做審查；
人定義規格、做判斷

每一次自動化往前推，人的工作就**往上移一層**。

CI/CD 能成功，靠的是 關卡

```
$ git push origin feature/close-ticket

build                PASS
unit test            FAIL
security scan        SKIPPED

merge                BLOCKED
```

測試失敗，就不能合併。

規則寫進 pipeline，每次都執行，沒有人能跳過。我們不靠每個人記得跑測試。

AI 寫程式之後，需要的是同一件事：
coding 階段的關卡。

瓶頸，從「寫」移到了「確認」

我的一次開發經驗，兩週

5.3 萬行

平均每天約 3,800 行（以 14 天計）

你一天能認真 review 幾行？

需求理解對嗎？架構放對層了嗎？
測試真的有測到嗎？有沒有安全問題？

人工逐行審查，已經跟不上這個速度。

工程師的角色也跟著改變

過去

寫程式的人

- 讀需求，自己寫程式和測試
- 送 review，等意見再修正
- 品質靠個人經驗與自覺

現在

設計交付流程的人

- 定義規格：讓 AI 有依據
- 設計規則與關卡：讓 AI 過不去就停
- 判斷例外與風險，最後 Approved

寫程式的能力還是需要，只是更多時間花在**讓 AI 寫對**。

真人團隊，本來就有一套交付流程

SA 寫規格 → 開發 → Code Review → 修正 → 再次 Review → 交付

有問題就退回修正，修完再審，**沒問題才交付**。資安檢查多半不在這條線上，而是交給 CI 上的 Fortify 掃描。

有依據

開發和 Review 都對照同一份規格

有關卡

沒過 Review 就回到開發

有分工

寫的人和審的人分開

每個真人角色，都有對應的 Agent 與關卡

真人團隊	Agent Team	怎麼確保做到
SA 寫規格，使用者確認	OpenSpec：spec／tasks	人確認後才開工
開發者寫程式、自測	Coding Agent ＋ Skill	Hook：測試沒過不准收工
團隊開發規範	team-rules：ARCH／NAME／SEC	建置規則與架構測試
Code Review	Code Review Agent（唯讀）	工具權限＋回報契約
修正後再次 Review	team-pipeline 流程 Skill	FAIL 退回，修完全部重審
CI 的 Fortify 掃描	Fortify Preverify Agent	提前到開發階段
Approved、commit／push	人	最後決定權留在人

為什麼用 Team，不用一個 Agent

一個 Agent 自己寫、自己測、
自己審，就是

球員兼裁判

上下文隔離
Reviewer 不帶著 Coding 的假設，從 spec 重新核對

最小權限
Reviewer 唯讀，只有 Coding 能改檔

專精、按需載入
每個角色只讀自己的 Skill 與 references

各自負責
結果回到主 Agent，FAIL 有明確的退回對象

代價　token 變多、要協調、交接可能遺漏。只改一行文案這種小修改，**一個 Agent 就夠**。

交付可信的兩根支柱

有依據 **Spec**

AI 照已確認的接受條件寫，不對需求的猜測寫

有關卡 **Harness**

把「請你遵守」變成「沒做到就過不去」

控制強度：越往右，模型越跳不過

文字規範
AGENTS.md、Skill

→

角色權限
Reviewer 唯讀

→

Hook
測試沒過不准收工

→

建置規則
命名、警告、架構測試

→

人
規格確認、最後 Approved

設計原則 **能用工具判定的，就不要交給模型判斷。**

一個主 Agent，三個專責角色

人
確認需求、最後 Approved

→

OpenSpec
spec、tasks，人確認後開工

→

主 Agent
team-pipeline

▼ 依序派工，每個角色的結果都回到主 Agent 判斷 ▼

Coding Agent
mvc-coding
可改檔、可執行；Hook 驗證

Code Review Agent
code-review
唯讀，只能跑唯讀 git

Fortify Preverify
fortify-preverify
唯讀，對照正式掃描分類

共用依據　AGENTS.md ＋ team-rules（ARCH／NAME／SEC）＋ 建置規則與架構測試

team-pipeline：用一個流程 Skill 串起所有角色

主 Agent 載入 team-pipeline	① 派工 Coding	② 自己再跑 verify	③ 派審 Code Review	④ PASS 才派 Fortify	⑤ FAIL 退回 Coding，全部重跑
Agent 檔 誰做、權限	coding-agent 可改檔、可執行；Stop hook 跑 verify	code-reviewer 唯讀；只放行唯讀 git	fortify-reviewer 唯讀；只放行唯讀 git		
工作者 Skill 怎麼做	mvc-coding 照 tasks 實作與測試	code-review 自己看 diff，對照 Spec 與規範	fortify-preverify 追資料流，對照 Fortify 分類		
團隊規範 做到什麼程度	CORE · ARCH · NAME · CODE · TEST · SEC	CORE · ARCH · NAME · CODE · TEST ＋ Spec Spec	SEC ＋ Fortify 正式分類		

串接規則　主 Agent 只載入 team-pipeline；子 Agent 依 Agent 檔的路徑讀自己的 Skill，結果照回報契約回到主 Agent。

讓 AI 照接受條件寫，而非照猜測寫

```
### Requirement: 結案工單
系統 SHALL 只允許 Open 工單結案為 Closed。
#### Scenario: 重複結案
- WHEN 對已結案的工單再次送出結案
- THEN 回應 409，狀態維持 Closed ①
— tasks.md —
- [ ] 2.1 Domain：Ticket.Close() 拒絕重複結案
- [ ] 2.2 單元測試：重複結案被拒絕 ②
- [ ] 3.1 整合測試：重複結案回 409 ②
```

示意：依 Workshop 工單需求撰寫的 spec.md、tasks.md 節錄

① 寫成可驗證的情境

WHEN／THEN 寫出回應碼與狀態。沒寫的，AI 只能自己決定是 404、409 還是 500。

② tasks 對到 tests

每條情境都有對應的測試工作；Domain 與 HTTP 各測各的責任。

③ 人確認後才開工

規格確認是停止點。Coding 與 Reviewer 對照的是同一份已確認的依據。

Spec 的兩個坑

OpenSpec 的三個動作　**propose**　產生規格與 tasks → **apply**　叫 AI 照 tasks 實作 → **archive**　併入系統現況規格

```
operations:
  apply:
    guidance:
      - Do not implement in the current context.
      - Start the team-pipeline skill. ①
  archive:
    guidance:
      - Archive only after the user accepted. ②
$ npx openspec archive build-ticket-board
```

節錄自 openspec/config.yaml（文字略為精簡）

① 有捷徑繞過流程

OpenSpec　apply 會讓單一 Agent 寫到底；用 guidance 導回 team-pipeline。

自己的機制　實作入口只留 team-pipeline，範本與 AGENTS.md 寫明，不給「照 spec 直接寫」的捷徑。

② 規格沒有累積成現況

OpenSpec　change 只記差異；驗收後由人 archive，併入 openspec/specs。

自己的機制　分開「本次變更」與「系統現況」兩種文件，交付後把變更併回現況。

Agent：權限就是邊界

```
---
name: code-reviewer
description: 唯讀檢查規範符合度、Spec 與測試。
tools: [read, search, execute] ①
agents: [] ②
hooks:
  PreToolUse:
    - command: "... reviewer-git-only.ps1" ①
---
先讀 AGENTS.md，再讀 code-review/SKILL.md ③
```

節錄自 .github/agents/code-reviewer.agent.md（Copilot 格式）

① 用權限限制，不靠叮嚀

tools 只給需要的。Reviewer 的 execute 由 hook 限定為唯讀 git，寫「請勿修改」擋不住。

② 子 Agent 不再派工

agents: [] 讓它不能自己再開子 Agent，派工只由主 Agent 決定。

③ Agent 檔要薄

只定角色、權限、指向哪個 Skill。方法放 Skill，三個工具共用同一份。

方法、標準、需求，分開放

方法 · Skill

怎麼做

2. 依架構審查方法，核對 ARCH 規範

.agents/skills/

標準 · 團隊規範

做到什麼程度

ARCH-05：禁止在 Controller 寫資料查詢

docs/team-rules/

需求 · Spec

這次要做什麼

重複結案，回應 409

openspec/changes/

換專案，換規範；換需求，換 Spec；**Skill 不動**。Skill 引用規則編號，不自己寫一份標準，也不猜業務需求。

Skill 設計的三個坑

```
---
name: code-review
description: 唯讀檢查 C# 程式變更是否符合
  團隊命名、架構、程式品質及測試規範，
  並核對本次 Spec。
  實作或修復完成、準備交付時使用。 ①
disable-model-invocation: true ②
---
# Code Review
2. 依 [架構審查方法](references/ ③
   architecture-review.md) 核對 ARCH 規範
```

節錄自 .agents/skills/code-review/SKILL.md

① description 決定誰會載入它

「準備交付時使用」對主 Agent 也成立，它會在同一個上下文裡自己審自己。

② 工作者 Skill 關閉自動觸發

Reviewer 子 Agent 依路徑直接讀 SKILL.md，不受影響；主 Agent 就不會順手拿來用。

③ 方法寫在 Skill，標準留在規範

細節放 references 用到才讀；引用 ARCH 規則編號，不在 Skill 裡再寫一份標準。

測試沒過，Coding 不准收工

```
hooks:
  Stop:
    - type: command
      command: "pwsh … eng/verify-on-stop.ps1" ①
      timeout: 900
— 未通過時，hook 回傳 —
{"decision": "block",
 "reason": "verify 未通過（第 1 次）…" } ②
```

節錄自 coding-agent.agent.md 與 eng/verify-on-stop.ps1 的輸出

① 由 Harness 執行，模型跳不過

Coding 要結束時，工具自動跑 restore、build、test，沒過就把錯誤帶回去繼續修。

② 不限次數，但要有出口

沒過就一直擋到通過；Coding 回報 status: BLOCKED 就放行，交給主 Agent 找人處理。

③ 只掛在 Coding

寫在 Coding 的 Agent 設定裡，只有它結束時才觸發；Reviewer 不受影響。

Reviewer 要 自己看 diff

1. 讀團隊規範與核准的 change，自行執行
   `git status --porcelain=v1 --untracked-files=all`
   `git diff HEAD`
   取得實際變更，不以 Coding 的清單代替。 ①

節錄自 code-review/SKILL.md ／下方為 finding 示意

status: FAIL
- [阻擋] ARCH-07 · Ticket.Close() ②
  只在 Controller 檢查狀態，其他入口可重複結案
  修正：由 Domain 拒絕重複結案，補單元測試

① 不信 Coding 的清單

Reviewer 用唯讀 git 自己取得變更。Coding 漏報的檔案，Reviewer 一樣看得到。

② finding 要能直接拿去修

規則編號、程式位置、證據、修正要求。只寫「不符合 DDD」，Coding 不知道要改哪裡。

把 CI 的安全掃描提前

正式 Fortify 分類	團隊規則	常見觸發點
Cross-Site Scripting: Reflected	SEC-02	Html.Raw 輸出使用者輸入
Log Forging	SEC-10	使用者輸入直接寫進日誌
Cookie Security: HTTPOnly not Set	SEC-11	Cookie 未設 HttpOnly
Denial of Service: Regular Expression	SEC-12	處理使用者輸入的 Regex 未設逾時

可證實問題
違反 SEC 規則，阻擋

預期誤報
附抑制說明草稿，不阻擋

需補資訊
安全邊界未確認，BLOCKED

不適用
沒有該資料流

分類名稱查證自 vulncat.fortify.com（2026-10-11）。預檢由模型分析原碼，不等於正式掃描。

回報契約：BLOCKED 是合法出口

```
status: PASS 或 FAIL 或 BLOCKED
reviewer: code-reviewer
## Scope       實際檢查的範圍
## Checks      規範 / Spec → 程式證據
## Findings    規則、位置、修正要求
## Limitations 未驗證的部分
```

節錄自 team-pipeline/references/report-contract.md

PASS

完成實際檢查，而且沒有阻擋問題。

FAIL

退回 Coding。修完後 build／test 與兩道審查**全部重跑**，不沿用修改前的 PASS。

BLOCKED

缺依據、環境問題、規格矛盾。停下來找人，不硬說 PASS。

防作弊機制：不能用改規則來過關

— AGENTS.md —

- 未經授權不得改團隊規範、Skill、Agent、eng/ 驗證腳本、hook 設定、建置設定或架構測試來消除失敗。

— docs/team-rules/testing.md —

- TEST-07：禁止刪除、跳過或弱化測試來消除 FAIL。

節錄，文字略為精簡

常見的捷徑 → 擋法

刪掉或弱化失敗的測試

→ TEST-07；Reviewer 核對被刪、被略過的測試

加 NoWarn、#pragma 關掉警告

→ CODE-07，Reviewer 視為阻擋問題

改 hook 或規範，讓流程變綠

→ AGENTS.md 明文禁止；Codex 沙箱下 .agents 為唯讀



全域放方法，專案放標準

層級	放什麼	例子
全域（個人、組織）	跨專案通用的方法	審查方法、安全預檢方法
專案	這個 repo 的標準與事實	AGENTS.md、team-rules、架構測試、Hook
專案內的路徑	前端、後端各自的規則	依目錄或檔案路徑套用的規則檔
本次變更	這次的需求	OpenSpec 的 spec、tasks

越往上越穩定、越能共用；越往下越常變動。放錯層，就會在每個專案各改一次。

全域 Skill 位置：~/.agents/skills（Codex、VS Code）、~/.claude/skills（Claude Code）

專案不一樣，換專案層，Skill 不動

同時有前端和後端

依路徑套用不同規則，前端可另加自己的 Coding Skill，審查方法照用。

VS Code	.github/instructions 的 applyTo
Claude	子目錄 CLAUDE.md、.claude/rules
Codex	根目錄到工作目錄的 AGENTS.md

不用 DDD 的專案

換掉 architecture.md 的分層規則與架構測試；mvc-coding、code-review 的方法不動，因為它們只引用規則編號。

前提：Skill 裡**不能寫死**某一種架構。

這套做法，也是修出來的

v1 文字規範與角色
規範、Skill、三個角色。發現：寫了，模型不一定照做

v2 加上 Hook
測試沒過不准收工；Reviewer 自己看 diff

v3 加上建置規則與分類對照
命名、警告、架構測試；Fortify 正式分類

從小專案打模

① 挑一個小專案

② 完整跑一次流程

③ 看 AI 在哪裡出錯

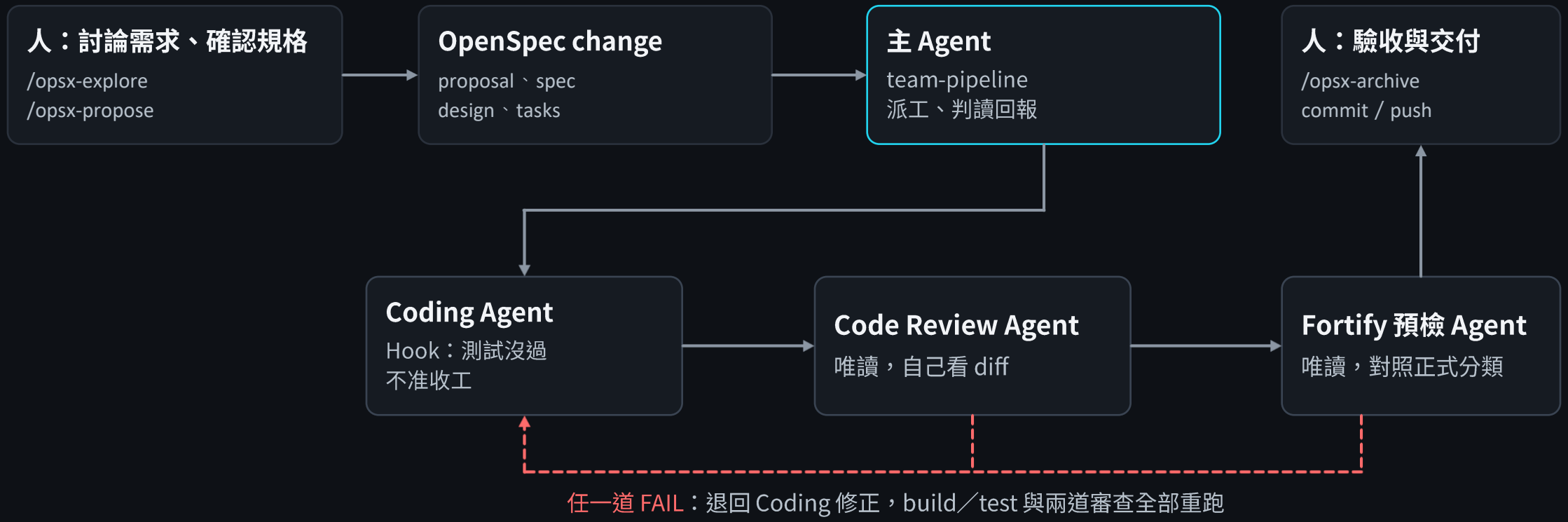
④ 補一道依據或關卡

⑤ 再跑一次

每個錯誤，都是下一道關卡的位置

> 總覽

一張圖看懂 AI Agent Team



> 總結

得到什麼、付出什麼：坦白說 取捨

得到：工程嚴謹度

能用工具判定的，交給工具
Hook、建置規則、架構測試每次結果一致，模型跳不過

寫的人和審的人分開
Reviewer 有獨立上下文、唯讀，自己看 diff

每個判斷都有依據
Spec 接受條件、規則編號、PASS／FAIL／BLOCKED

關鍵決策留給人
規格確認、例外判斷、Approved 與 commit／push

付出：成本與限制

Token 與時間變多
三個子 Agent 各自讀規範與程式；FAIL 後全部重審

協調成本增加
主 Agent 要派工、判讀、轉交 findings，多一層出錯點

前期設定與持續維護
規範、Skill、Agent、Hook 要寫，也要跟著專案演進

模型判斷仍會漏
Review 與預檢是模型判斷，不取代正式掃描與人審

何時值得　需求明確、要交付可信、會重複的開發工作；小修改或探索原型，一個 Agent 就夠。

Q&A

簡報

臉書

自動化往前推一段，關卡也要跟著往前；
人把時間留給規格、例外與判斷。

陳葵懋 Ian Chen

LAB

想動手做？AI Coding 治理實戰：從開發到交付的 Harness 工程

90 分鐘 Workshop，帶筆電，現場建立並跑一次完整的 Agent Team